DAA PRACTICALS

Easy Explanations & Oral Viva Questions with Answers

Included practicals: Fibonacci Series, Huffman Encoding, Fractional Knapsack, 0-1 Knapsack and N-Queens.

Prepared from the five practical PDFs provided in this chat. Explanations use simple, exam-friendly language.

How to prepare for the viva

For each practical, remember the aim, the main idea, the important steps and the key Python functions. You do not need to memorize every line. Be ready to explain what each important line does.

Note: The 0-1 Knapsack program in your practical uses Dynamic Programming. The N-Queens program places the first Queen at a position entered by the user, then searches for valid positions for the remaining Queens.

Practical 1: Fibonacci Series

Aim: Write recursive and non-recursive programs to calculate Fibonacci numbers and analyse their time and space complexity.

Easy explanation

The Fibonacci series is a sequence in which each number is the sum of the two numbers before it.

Example: 0, 1, 1, 2, 3, 5, 8, 13, 21

Recursive and non-recursive methods

Recursive	Non-recursive
A function calls itself.	Uses a loop instead of calling itself.
The function stops at a base condition.	The loop stops when its condition becomes false.
In this program: O(2■) time and O(n) space.	In this program: O(n) time and O(1) space.

Program flow

• Non-recursive: start with n1 = 0 and n2 = 1; print n1; calculate the next number as n1 + n2; update the two variables; repeat.

• Recursive: if n is 0 or 1, return n. Otherwise return fibonacci(n-1) + fibonacci(n-2).

• The base condition is if n <= 1. It stops the recursive calls.

Complexity to remember: Non-recursive: Time O(n), Space O(1). Recursive version shown: Time O(2■), Space O(n).

Oral viva questions and answers

Q. What is the Fibonacci series?
Ans. A series in which each number is the sum of the previous two numbers.

Q. What is recursion?
Ans. A function calling itself to solve a smaller part of a problem.

Q. What is non-recursion?
Ans. Solving a problem without a function calling itself, usually with loops.

Q. What is a base condition?
Ans. The condition that stops recursive calls.

Q. What is the base condition in this program?
Ans. if n <= 1.

Q. Which loop is used in the non-recursive program?
Ans. A while loop.

Q. What is the time complexity of the non-recursive version?
Ans. O(n).

Q. What is the space complexity of the non-recursive version?
Ans. O(1).

Q. What is the time complexity of the recursive version shown?
Ans. O(2■).

Q. Which method is more efficient in this program?
Ans. The non-recursive method, because it uses less time and memory.

Practical 2: Huffman Encoding

Aim: Implement Huffman Encoding using a Greedy Strategy.

Easy explanation

Huffman Encoding is a lossless data-compression technique. It gives shorter binary codes to characters that appear more often and longer codes to characters that appear less often.

Your practical uses these frequencies: A = 6, B = 7, C = 2, D = 23, E = 22 and F = 47.

How the program works

• Import heapq. It helps create a priority queue (min-heap).

• Create a Node that stores a character, its frequency, and left and right child nodes.

• Insert each character with its frequency into the heap.

• Remove the two nodes with the smallest frequencies and combine them into a new node whose frequency is their sum.

• Insert the combined node back into the heap. Repeat until one node remains. This is the root of the Huffman Tree.

• Generate codes by following the tree: left branch adds 0 and right branch adds 1.

• Calculate the total encoded bits and average code length.

Important terms

Term	Simple meaning
Greedy strategy	Choose the best available option at each step.
Frequency	How many times a character occurs.
Heap / priority queue	Keeps the smallest-frequency node ready to remove.
Huffman Tree	Binary tree used to generate the codes.
Lossless compression	Original data can be recovered without losing information.

Remember: The exact bit code can depend on how equal-frequency nodes are ordered. The important rule is to repeatedly combine the two smallest frequencies.

Oral viva questions and answers

Q. What is Huffman Encoding?
Ans. A lossless data-compression technique that uses variable-length binary codes.

Q. Which strategy is used?
Ans. Greedy strategy.

Q. Why is it called lossless compression?
Ans. Because the original data can be recovered without losing information.

Q. Which Python module is used in your program?
Ans. heapq.

Q. What is a priority queue?
Ans. A data structure that removes elements according to priority. Here, the smallest frequency is removed first.

Q. What does a Node store?
Ans. Frequency, character, left child and right child.

Q. What is a Huffman Tree?
Ans. A binary tree used to generate Huffman codes.

Q. What do 0 and 1 represent in your code generation?
Ans. 0 represents the left branch and 1 represents the right branch.

Q. Which characters generally get shorter codes?
Ans. Characters with higher frequencies.

Q. What is the root node?
Ans. The final combined node that represents all characters.

Q. What is the purpose of __lt__?
Ans. It compares two nodes by their frequencies so heapq can order them.

Practical 3: Fractional Knapsack

Aim: Solve the Fractional Knapsack problem using a Greedy Method.

Easy explanation

Imagine a bag with a fixed weight capacity. Each item has a weight and a value. The aim is to get the maximum value without exceeding the bag's capacity. In Fractional Knapsack, you may take a complete item or only part of an item.

Main formula

Value-to-weight ratio = Value / Weight
Value of a fraction = (Item value × remaining capacity) / item weight

Example

Item	Weight	Value	Value / Weight
1	5	100	20
2	5	200	40
3	10	300	30

Choose items in descending order of ratio: Item 2 first, then Item 3, then Item 1. If only part of the next item fits, take that fraction.

Program flow

• Read the number of items, values, weights and bag capacity W.

• Use sorted() with the key value/weight and reverse=True to sort from highest ratio to lowest.

• Take a full item if its weight is less than or equal to the remaining capacity.

• If it does not fit completely, take the fraction that fits and set the remaining capacity to zero.

• Return the total value.

Oral viva questions and answers

Q. What is the Fractional Knapsack problem?
Ans. A problem where complete items or fractions of items can be selected to maximize value within a capacity.

Q. Which method is used?
Ans. Greedy method.

Q. What is the objective?
Ans. To get the maximum total value without exceeding the capacity.

Q. How do you calculate the ratio?
Ans. Divide the item's value by its weight.

Q. Why are items sorted?
Ans. To consider the highest value-to-weight ratio first.

Q. Can we take half an item?
Ans. Yes. Fractions are allowed.

Q. What does sorted() do?
Ans. It returns the items in sorted order.

Q. What does reverse=True do?
Ans. It sorts in descending order.

Q. What happens when the remaining capacity is zero?
Ans. The loop stops because no more weight can be added.

Q. What is the difference between Fractional and 0-1 Knapsack?
Ans. Fractional allows part of an item; 0-1 allows only the whole item or none.

Practical 4: 0-1 Knapsack

Aim: Solve the 0-1 Knapsack problem using Dynamic Programming or Branch and Bound. The uploaded program uses Dynamic Programming.

Easy explanation

A bag has a fixed capacity. Every item has a weight and a value. For each item, you have only two choices: take the whole item (1) or do not take it (0). You cannot take a fraction.

How the program works

• Read the number of items, their values, their weights and capacity W.

• Create a DP table with n + 1 rows and W + 1 columns, initially filled with zero.

• The outer loop considers items. The inner loop considers capacities.

• If the current item's weight fits, compare two choices: include the item or exclude it.

• Store the larger value using max(). If the item does not fit, copy the value from the previous row.

• Return dp[n][W], the best value for all items and the full capacity.

Include choice: current item value + best value for the remaining capacity.
Exclude choice: best value without the current item.
Decision: choose the larger of the two.

Important terms

Term	Simple meaning
Dynamic Programming	Solve smaller subproblems and store their results.
DP table	A table that stores results for items and capacities.
max()	Returns the larger of the include and exclude choices.
Capacity	Maximum weight the bag can hold.

Complexity: Time O(n × W), Space O(n × W), where n is the number of items and W is the capacity.

Oral viva questions and answers

Q. What is 0-1 Knapsack?
Ans. A problem where each item is either selected completely or not selected.

Q. Why is it called 0-1?
Ans. Each item has two choices: 0 means exclude and 1 means include.

Q. Which technique does your program use?
Ans. Dynamic Programming.

Q. What is Dynamic Programming?
Ans. It solves smaller subproblems and stores their results to avoid repeated work.

Q. What is a DP table?
Ans. A table that stores results for different item counts and capacities.

Q. Can we take half an item?
Ans. No.

Q. What does max() do in the program?
Ans. It selects the larger value between including and excluding the item.

Q. Why are there two loops?
Ans. One loop goes through items and the other goes through capacities.

Q. What is the time complexity?
Ans. O(n × W).

Q. What is the space complexity?
Ans. O(n × W).

Q. What is the difference between Greedy and Dynamic Programming?
Ans. Greedy makes the best immediate choice; Dynamic Programming compares stored subproblem results.

Practical 5: N-Queens Problem

Aim: Design an N-Queens matrix with the first Queen placed, then use Backtracking to place the remaining Queens.

Easy explanation

Place N Queens on an N × N chessboard so that no two Queens can attack each other. No two Queens may share the same row, column or diagonal.

What is Backtracking?

Backtracking means trying a possible choice. If it leads to no valid solution, undo that choice and try another one.

Program flow

• Create an N × N board. In the program, False means an empty square and True means a Queen is placed there.

• Ask the user for the first Queen's row and column, then place it on the board.

• isSafe(row, col) checks whether a new Queen can be placed at that position.

• The safety check looks for another Queen in the same column and along both diagonal directions.

• solve(row) moves through the rows. If a position is safe, it places a Queen and calls solve() for the next row.

• If a later row cannot be completed, the program removes the last Queen by setting that board position back to False.

• When all rows are handled, the program prints the solution and increases the solution count.

Important: The program asks for the first Queen's position. If that position prevents a valid arrangement, it reports that no solution exists for the given position.

Important terms

Term	Simple meaning
N-Queens	Place N non-attacking Queens on an N × N board.
Backtracking	Undo a choice when it cannot lead to a solution.

Term	Simple meaning
isSafe()	Checks whether a square is safe for a Queen.
solve()	Searches for valid Queen positions row by row.

Oral viva questions and answers

Q. What is the N-Queens problem?
Ans. Place N Queens on an N × N chessboard so that no two Queens attack each other.

Q. Which technique is used?
Ans. Backtracking.

Q. What is Backtracking?
Ans. Trying a choice and undoing it if it does not lead to a valid solution.

Q. What are the three attack directions to check?
Ans. Same row, same column and diagonals.

Q. What is the use of isSafe()?
Ans. It checks whether a Queen can be placed safely at a given position.

Q. What does True mean in the board?
Ans. A Queen is present in that square.

Q. What does False mean?
Ans. The square is empty.

Q. What does solve() do?
Ans. It searches for valid positions row by row.

Q. What happens if a position is unsafe?
Ans. The program tries another column.

Q. What happens when a choice fails later?
Ans. The program removes that Queen and tries another position.

Q. What is a 4-Queens board size?
Ans. 4 × 4.

Q. What is a commonly stated upper bound for N-Queens backtracking time?
Ans. O(N!).

Quick comparison: the five practicals

Practical	Main concept / technique	Key point
1. Fibonacci	Recursion and iteration	Sum of previous two numbers.
2. Huffman	Greedy strategy	Combine the two smallest frequencies.
3. Fractional Knapsack	Greedy strategy	Choose the highest value/weight ratio.
4. 0-1 Knapsack	Dynamic Programming	Compare include and exclude.
5. N-Queens	Backtracking	Place Queens safely and undo failed choices.

15 quick viva questions

Q. What is an algorithm?
Ans. A step-by-step method to solve a problem.

Q. What is time complexity?
Ans. How the running time grows as the input size grows.

Q. What is space complexity?
Ans. How the memory use grows as the input size grows.

Q. What is recursion?
Ans. A function calling itself.

Q. What is a base condition?
Ans. The condition that stops recursion.

Q. What is Greedy Method?
Ans. Choosing the best available option at each step.

Q. What is Dynamic Programming?
Ans. Solving smaller problems and storing their results.

Q. What is Backtracking?
Ans. Going back and trying another choice when a choice fails.

Q. What is a priority queue?
Ans. A structure that processes elements according to priority.

Q. What is a heap?
Ans. A tree-based structure that maintains an ordering rule.

Q. What is Huffman Encoding?
Ans. A lossless data-compression method.

Q. What is a knapsack?
Ans. A capacity-limited bag used in an optimization problem.

Q. What is a DP table?
Ans. A table used to store subproblem results.

Q. What is an optimal solution?
Ans. The best solution among the valid choices.

Q. Which practical uses backtracking?
Ans. The N-Queens practical.

Before the viva, practise explaining the purpose of heapq, sorted(), max(), isSafe(), solve(), and the Fibonacci base condition.